

UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS
DEPARTAMENTO DE ENGENHARIAS E TECNOLOGIA

LABORATÓRIO DE CIRCUITOS DIGITAIS

Dispositivos Lógico-Programáveis e Linguagem de Descrição SystemVerilog

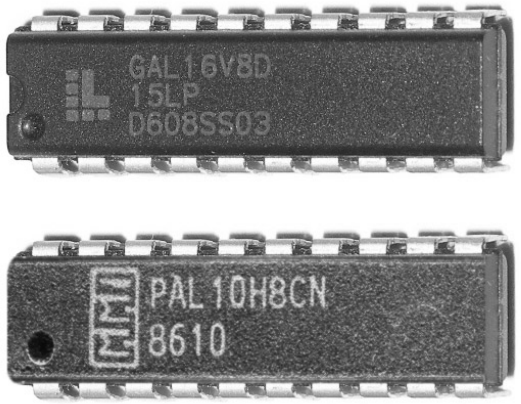
Prof.: Pedro Thiago Valério de Souza
UFERSA – Campus Pau dos Ferros
pedro.souza@ufersa.edu.br

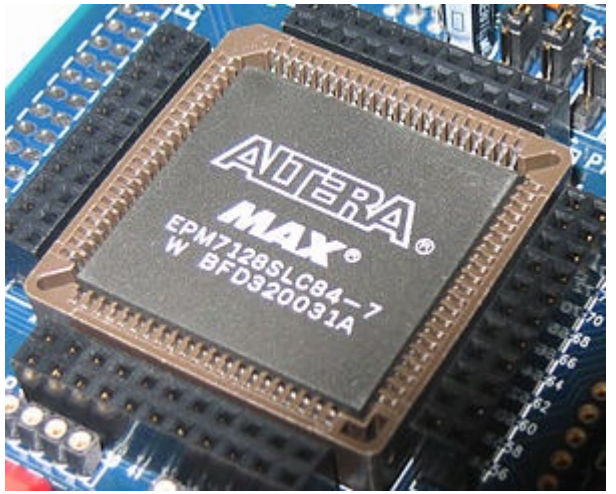
Implementação Física de Circuitos Digitais

- CIs de lógica padrão - *Standard Logic*;
- Circuitos integrados totalmente customizados – *Full-Custom IC*;
- Circuitos integrados específicos para aplicação – ASIC;
- **Dispositivo lógico programável – PLD;**
  - Dispositivo lógico programável simples – SPLD;
  - Dispositivo lógico programável complexo – CPLD;
  - **Arranjo de matriz de portas programável em um campo – FPGA.**

Implementação Física de Circuitos Digitais

- ***Programmable Logic Devices* (PLD):**
  - Todas as camadas do circuito já estão prontas;
  - Projetistas podem comprar os dispositivos programáveis e ambientes de desenvolvimento;
  - Conexões no CI são criadas ou destruídas para implementar as funcionalidades desejadas;



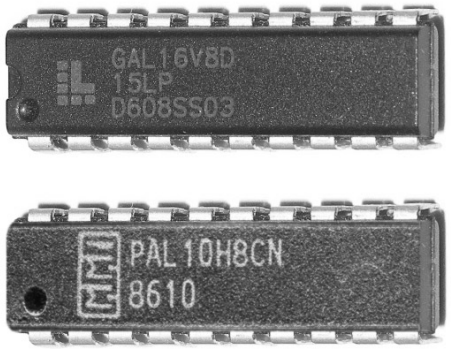


Implementação Física de Circuitos Digitais

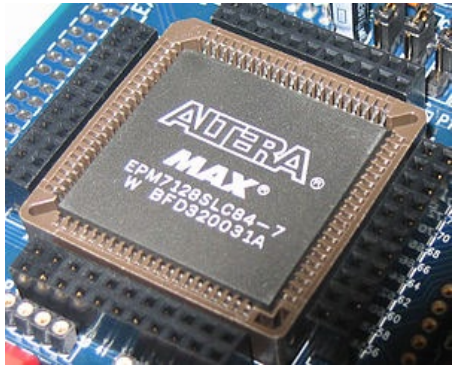
- ***Programmable Logic Devices* (PLD):**
  - *Vantagens:*
    - Baixo custo para projeto;
      - Permite a reutilização do dispositivo;
    - Rápido desenvolvimento;
    - Rapidez e facilidade de síntese e testes do circuito;
  - *Desvantagens:*
    - Grande tamanho físico;
    - Custos unitários altos;
    - Alto consumo;
    - Relativamente lentos.

Dispositivos Lógicos Programáveis

- **Principais arquiteturas de PLDs:**
  - SPLDs: Dispositivos Lógicos Programáveis Simples;
  - CPLDs: Dispositivos Lógicos Programáveis Complexos;
  - FPGAs: Arranjos de Gates Programáveis no Campo;



SPLD



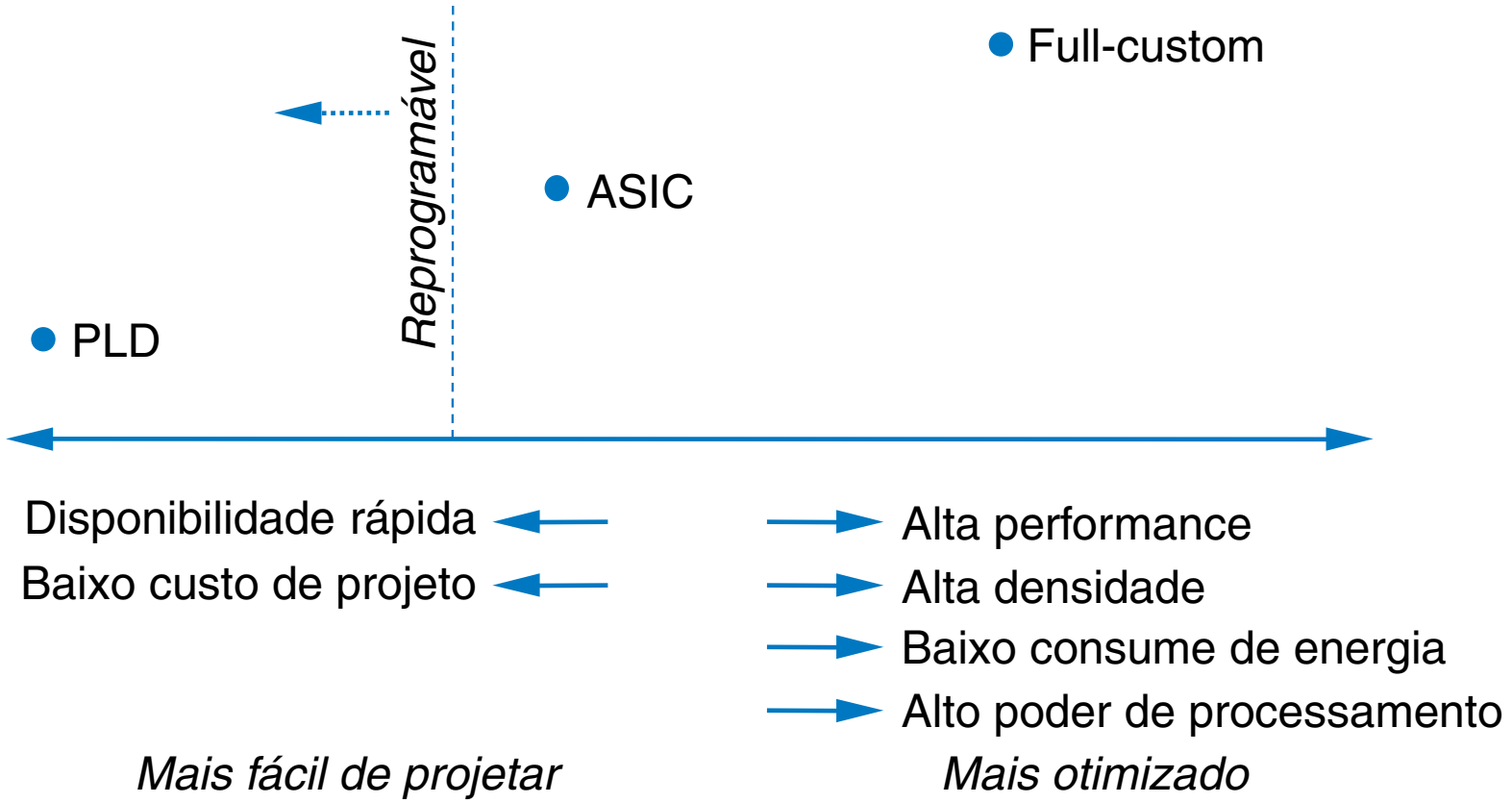
CPLD



FPGA

Implementação Física de Circuitos Digitais

- Relações de compromissos:



Linguagem de Descrição de *Hardware*

- Uma linguagem de descrição de *hardware* (HDL - *Hardware Description Language*) é a linguagem textual usada para descrever o *hardware* a ser sintetizado em uma FPGA/CPLD;
- Linguagens comuns:
  - AHDL (*Altera Hardware Description Language*);
  - VHDL (VHSIC - *Very High Speed Integrated Circuits - Hardware Description Language*);
  - Verilog/SystemVerilog;
  - System C;
- Dentre as linguagens de descrição de *hardware* destaca-se o SystemVerilog;

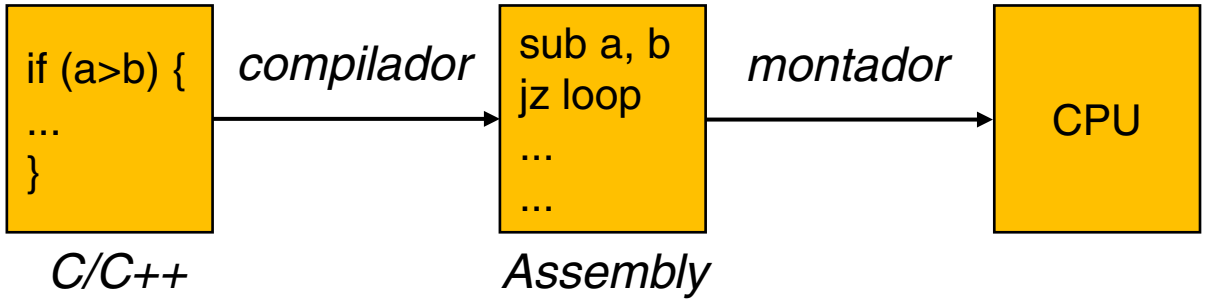
Linguagem de Descrição de *Hardware*

- **Vantagens na utilização do HDL:**
  - Projetos independentes da tecnologia (CPLD/FPGA);
    - Portabilidade;
  - Facilidade de atualização dos projetos;
  - Projeto em um nível mais alto de abstração;
  - Redução do tempo de projeto, de testes e implementação;
  - Simplificação quanto a sua documentação;

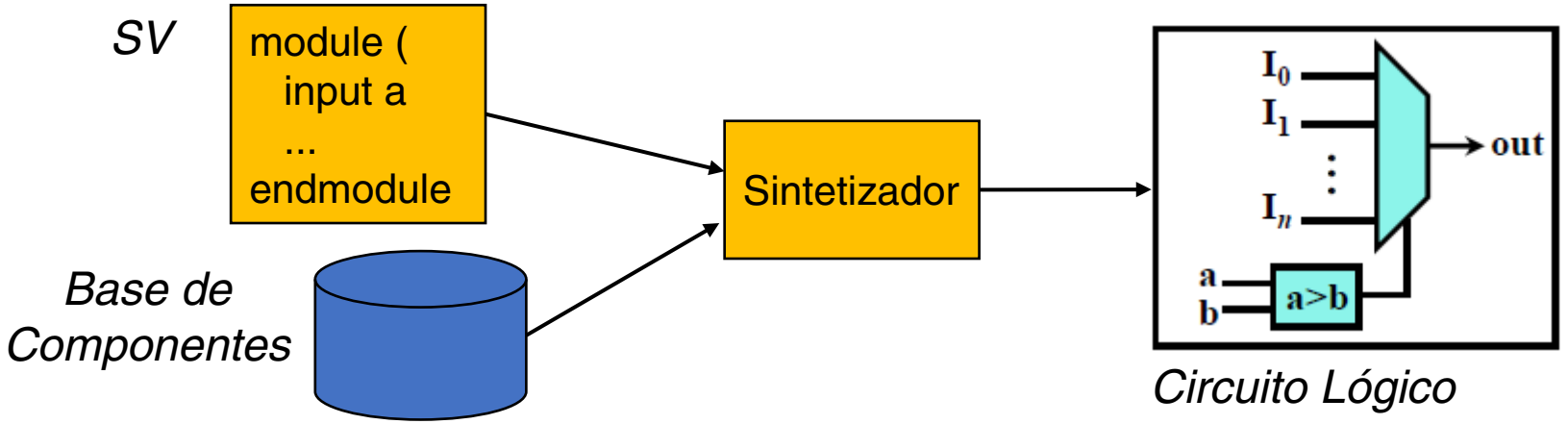
- **Desvantagens na utilização do HDL:**
  - O *hardware* gerado geralmente é menos otimizado.

Linguagem de Descrição de *Hardware*

- Linguagem de Descrição de *Hardware* vs. Linguagem de Programação.
  - Linguagem de Programação:

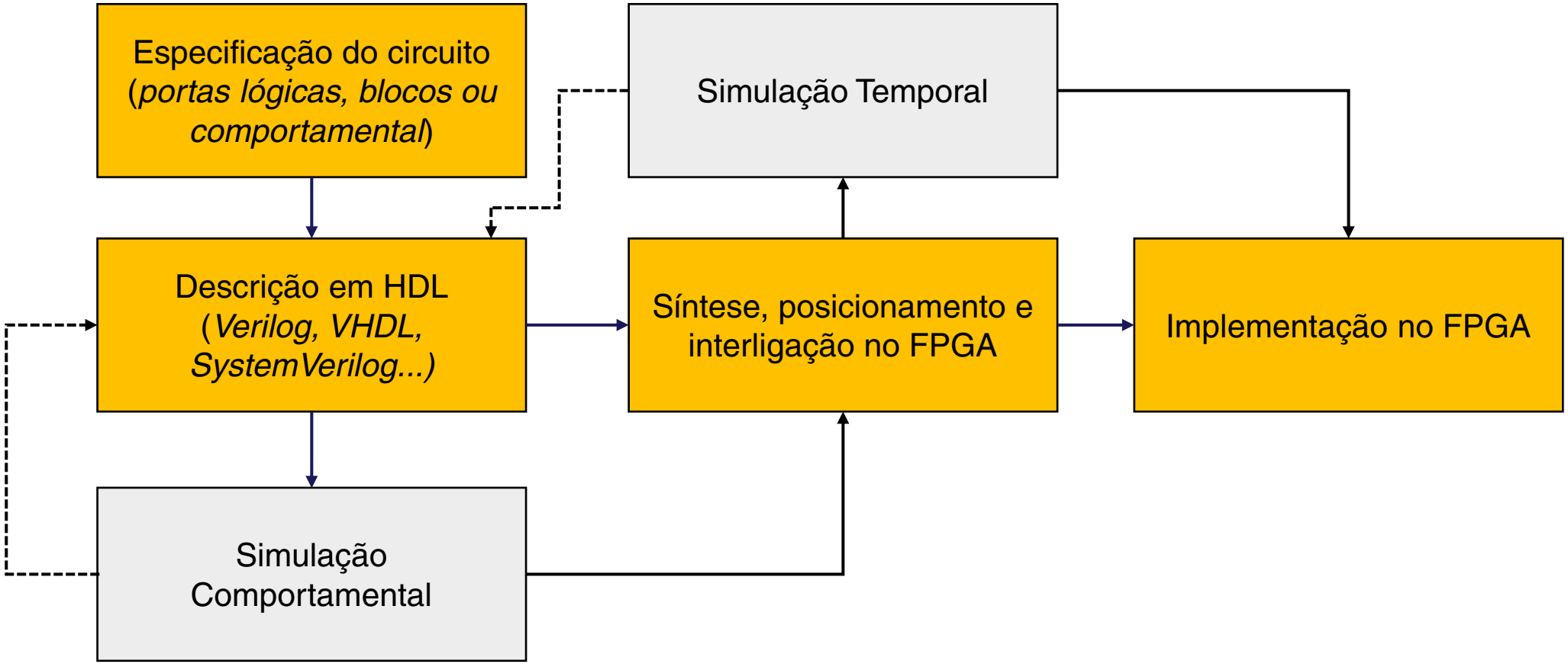


  - Linguagem de Descrição de *Hardware*:



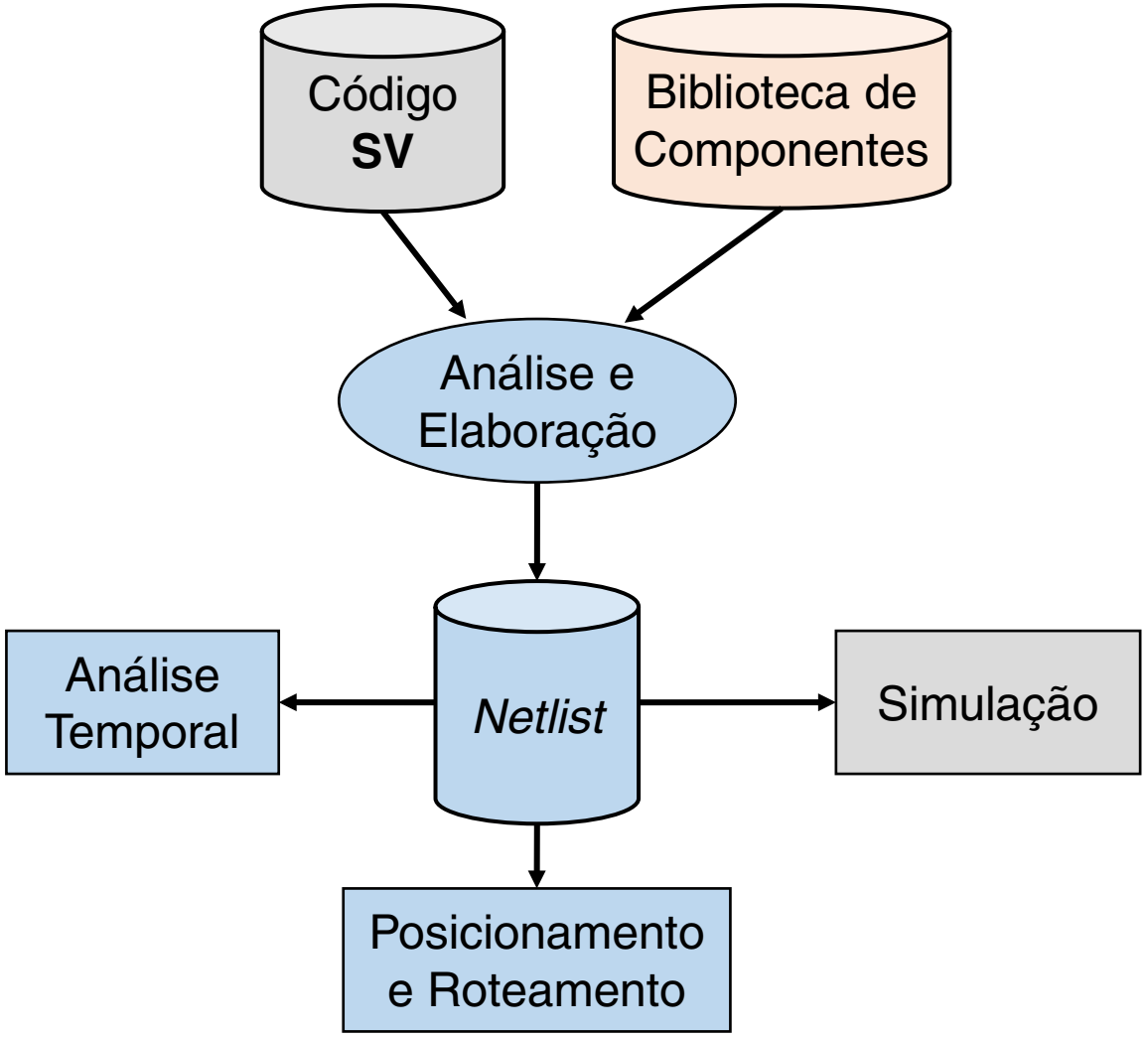
Linguagem de Descrição de *Hardware*

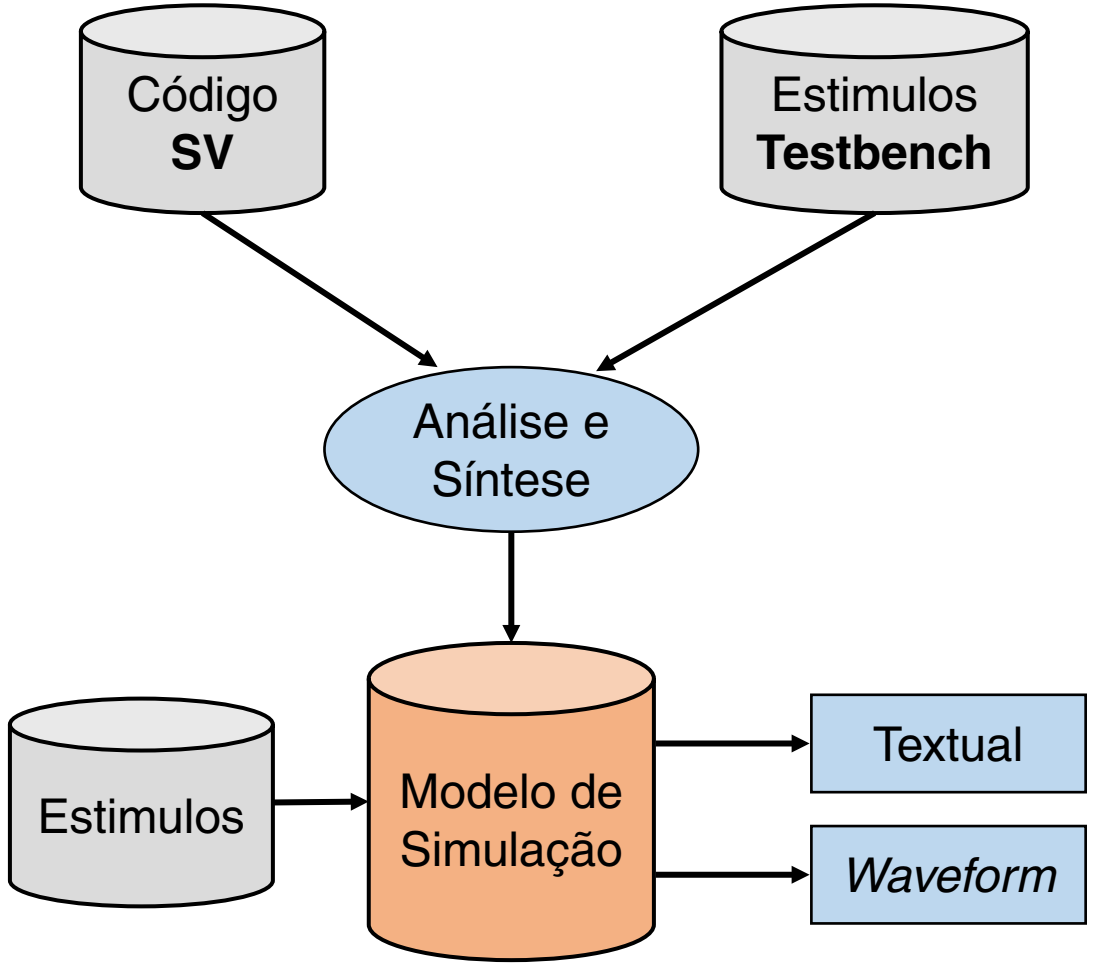
- Fluxo de Projeto:



Linguagem de Descrição de *Hardware*

- Fluxo de Projeto:





Estrutura Básica do SystemVerilog

```
module nome_module(lista_portas);

    Declaração das Portas

    Dados intermediários

    Funcionamento do Circuito

endmodule
```

- Inicia com **`module`** e encerra com **`endmodule`**;
- *Case-sensitive*;
- Todas as palavras chaves são minúsculas;
- Espaços em branco não são interpretados;
- Todos os comandos terminam em ponto-e-vírgula;
- //: Comentário em uma única linha;
- /* */: Comentário em múltiplas linhas;
- Regras para os identificadores: deve sempre iniciar com uma letra ou com _ , jamais deve iniciar com um número.

Declaração do Modulo (*module*)

- Inicia com a palavra **module**;
- Em seguida, vem o nome do modulo;
- Após isso, a lista de portas (se necessário).

```
module nome_module(lista_portas);
    Declaração das Portas
    Dados intermediários
    Funcionamento do Circuito
endmodule
```

Exemplo:

```
module halfadder (a, b, s, cout);
    (...)
endmodule
```

Declaração das Portas

- Comunicação do módulo com o mundo externo;
- Tipos de portas:
  - `input` → entrada;
  - `output` → saída;
  - `inout` → bidirecional.
- **Declaração de portas simples (1-*bit*):**

```
<tipo_da_porta> <tipo_do_dado> nome_da_porta;
```

Nem sempre é necessário!

```
module nome_module(lista_portas);
    Declaração das Portas
    Dados intermediários
    Funcionamento do Circuito
endmodule
```

Exemplo:

```
module halfadder (a, b, s, cout);
    input a, b;
    output s, cout;
endmodule
```

Declaração das Portas

- **Declaração de Barramentos (conjunto de *bits*):**

```
<tipo_da_porta> <tipo_do_dado> [MSB:LSB]
                nome_da_porta;
```

```
module nome_module(lista_portas);
    Declaração das Portas
    Dados intermediários
    Funcionamento do Circuito
endmodule
```

Exemplo:

```
module adder (a, b, cin, s, cout);
    input [3:0] a, b;
    input cin;
    output [3:0] s;
    output cout;
endmodule
```

Declaração das Portas

- **Definição de Constantes `(parameter)`:**
  - São valores numéricos que são associados a um nome;
  - Declaração:

```
parameter nome = valor;
```

  - Exemplos:

```
parameter barramento = 8;
```

  - Observação: A declaração de um `parameter` pode ser feita a qualquer momento no código Verilog;

Declaração das Portas

- **Definição de Constantes `(parameter)`:**
  - O uso de constantes pode tornar o código mais legível e mais fácil de atualizações;
  - Exemplo:

```
module adder (a, b, cin, s, cout);
    parameter largura = 4;
    input [largura - 1:0] a, b;
    input cin;
    output [largura - 1:0] s;
    output cout;
endmodule
```

Declaração do Módulo

- Após o Verilog-2001, a declaração do módulo e das portas podem ser combinadas;
- Permite a declaração de forma mais concisa;
- Exemplo:

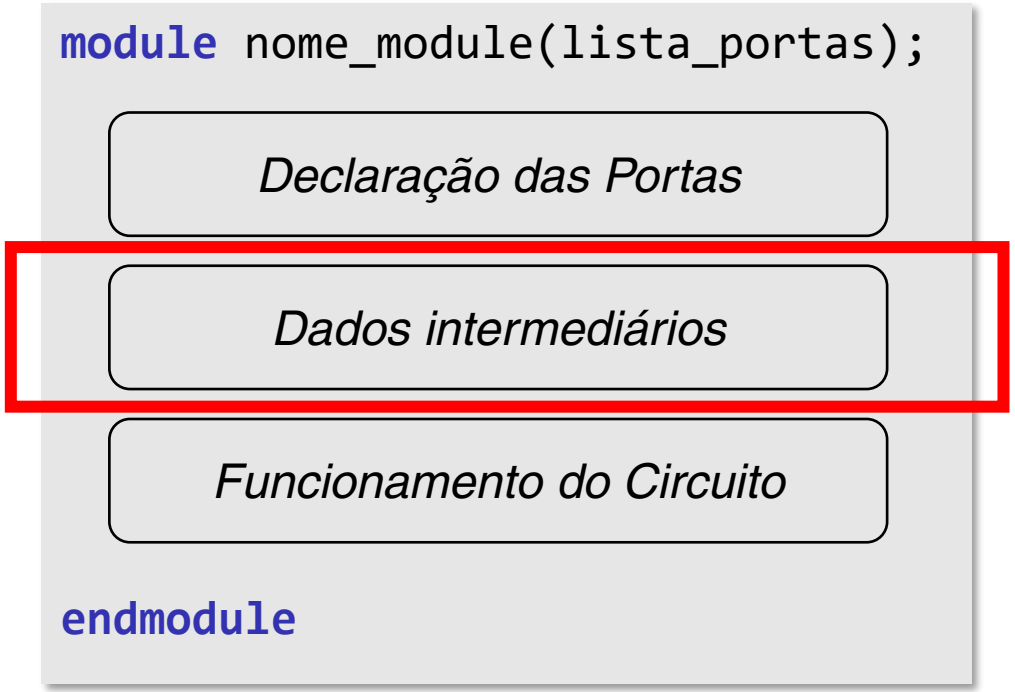
```
module adder
    #(parameter size = 8)
    (
    input [size - 1:0] a, b,
    input cin,
    output [size - 1:0] s,
    output cout
    );
endmodule
```

Separação por vírgulas (como se fosse o argumento de uma função).

A última linha não tem virgula.

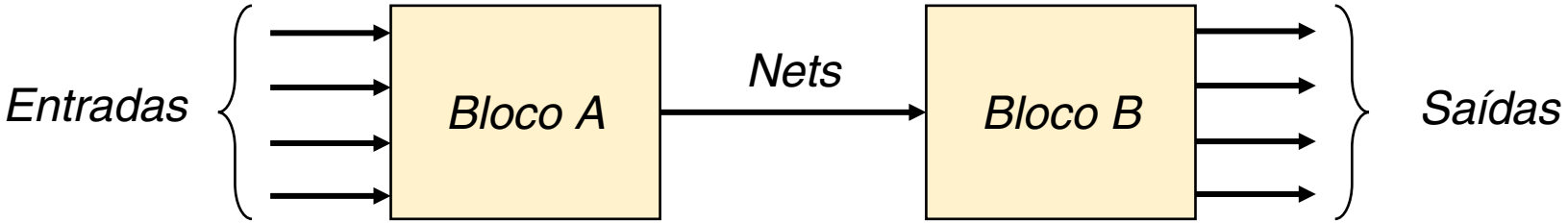
Dados Intermediários (*Data Type*)

- São dados internos ao módulo;
- Podem ser:
  - *Net Data Type*;
  - *Variable Data Type*.

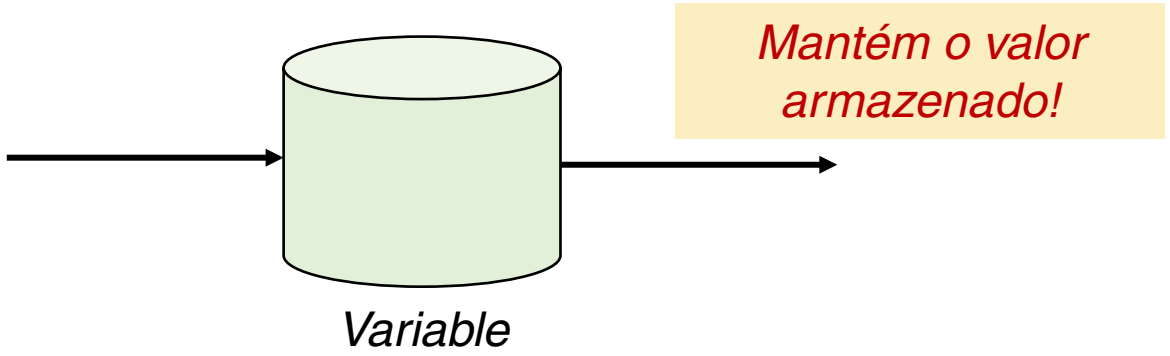


Dados Intermediários (*Data Type*)

- ***Net Data Type*:** Representam uma conexão física (fio) interno ao módulo;



- ***Variable Data Type*:** Representam um armazenamento local (registrador);



Dados Intermediários (*Data Type*)

- ***Net Data Type*:** Não apresentam capacidade de armazenamento;

Tipo	Definição
wire	Representa uma conexão (fio físico)

Exemplos:

```
wire ligacao;
wire [7:0] soma;
```

Observação: A declaração de Nets com mais de um bit é feita da mesma forma dos ports!

Dados Intermediários (*Data Type*)

- ***Variable Data Type*:** Apresentam capacidade de armazenamento;

Tipo	Definição
reg	Valor sem sinal
integer	Inteiro com sinal de 32-bits
real, time, realtime	Não sintetizáveis pelo Hardware

Exemplos:

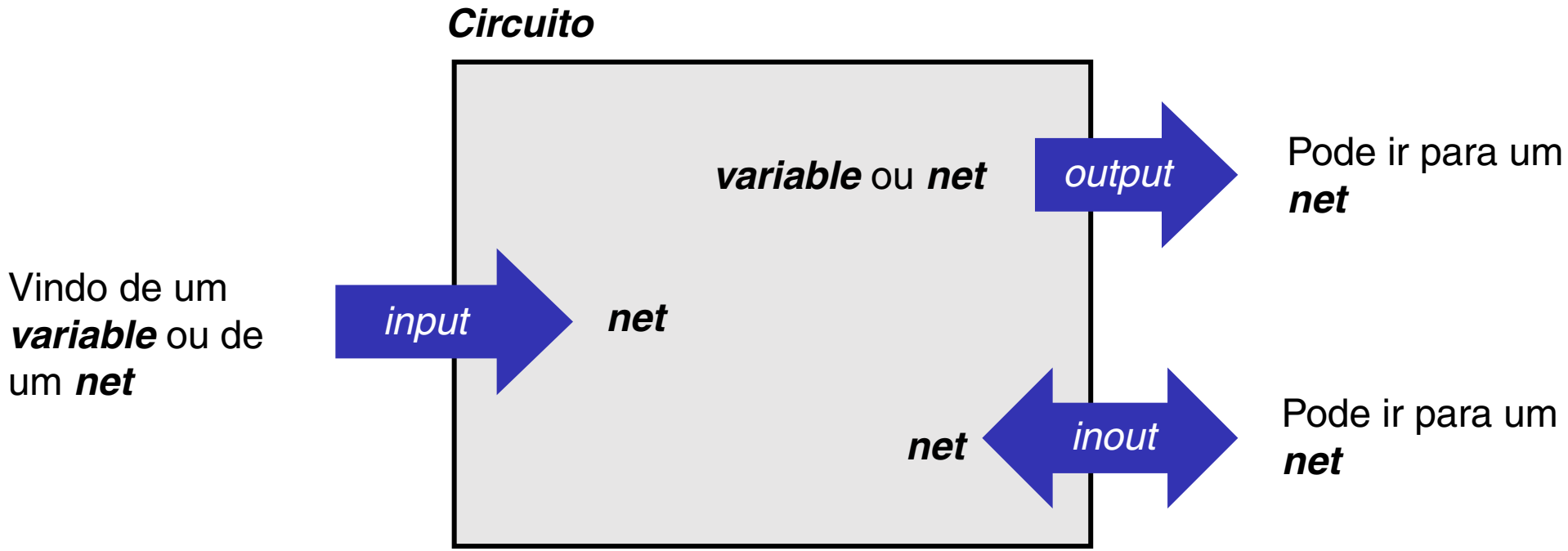
```
reg [7:0] resultado;
integer count;
```

Observação: A declaração de Register com mais de um bit é feita da mesma forma dos ports!

Dados Intermediários (*Data Type*)

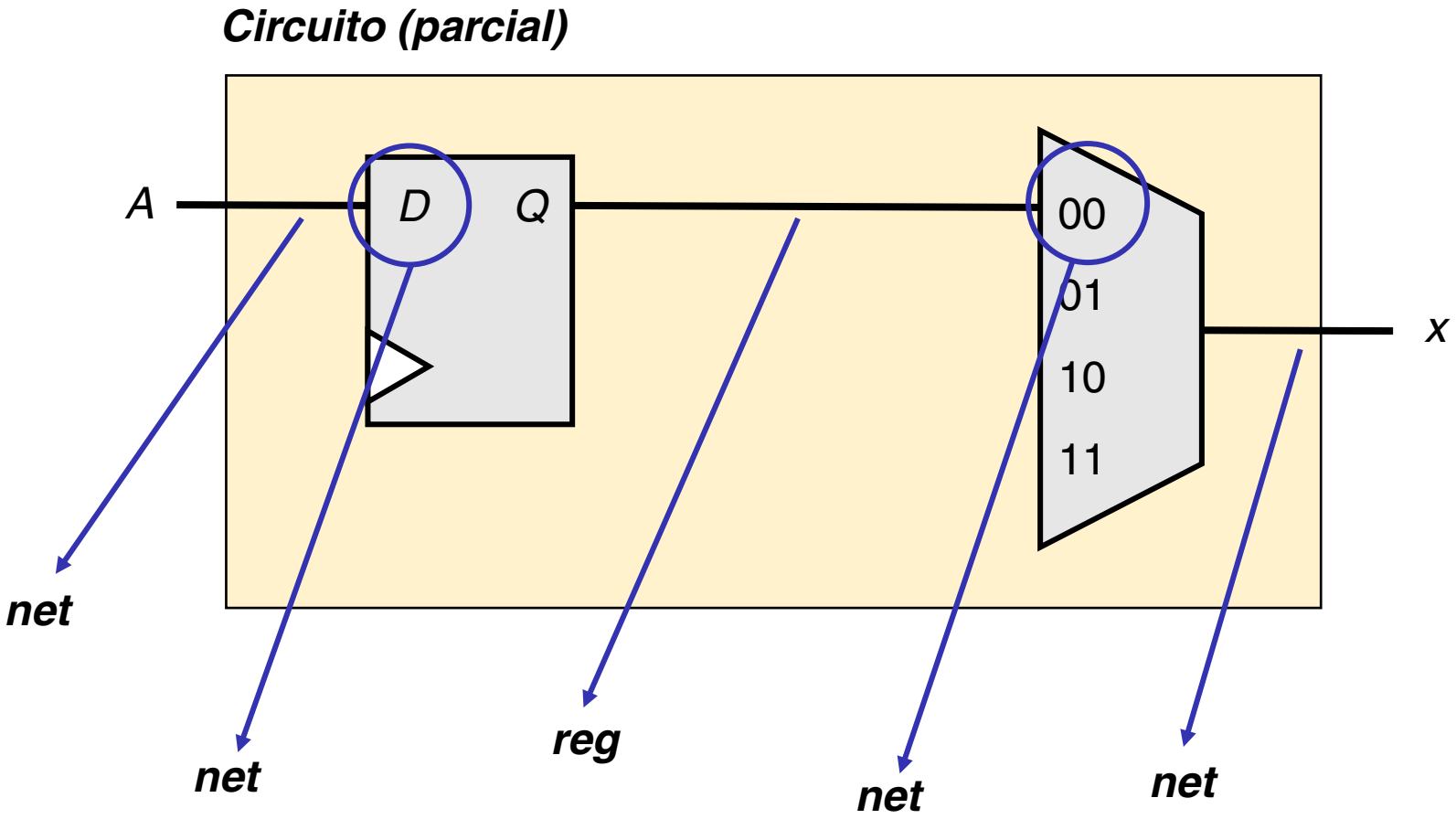
- **Regras para Entrada/Saída:**

Tipo de Variável	Entrada	Saída	IN/OUT
reg	NÃO	SIM	NÃO
wire	SIM	SIM	SIM



Dados Intermediários (*Data Type*)

- **Regras para Entrada/Saída:**
  - Exemplo:



Dados Intermediários (*Data Type*)

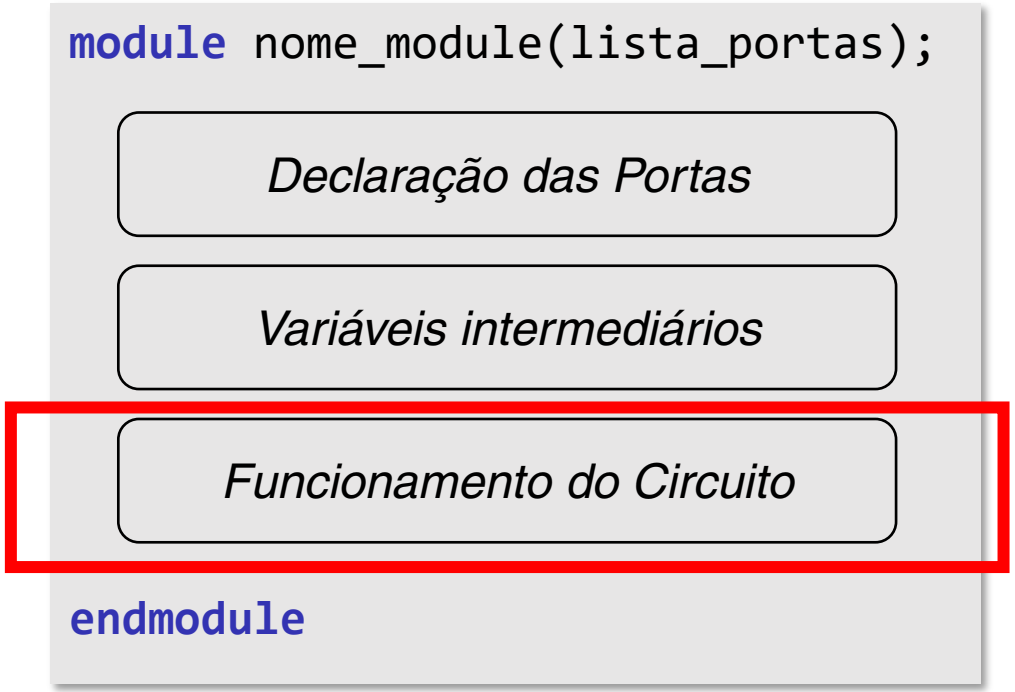
- Observação: Tipo `logic` e `bit`
  - Introduzido no SystemVerilog;
  - Consistem em dados do tipo *Variable data type*, mas que permitem que sejam utilizados como entrada.
  - Diferença entre o `logic` x `bit`:
    - `logic`: admite nível lógico alto, baixo, *don't care* e alta impedância;
    - `bit`: admite apenas nível lógico alto e baixo.

Dados Intermediários (*Data Type*)

	Tipo de Dado	#Estados	#Bits	signed/unsigned	Equivalente em C	
SYSTEM VERILOG	reg	4	>=1	unsigned		VERILOG
	wire	4	>=1	unsgined		
	integer	4	32	signed		
	real				double	
	time					
	realtime				double	
	logic	4	>=1	unsigned		
	bit	2	>=1	unsigned		
	byte	2	8	signed	char	
	shortint	2	16	signed	short int	
	int	2	32	signed	int	
	longint	2	64	signed	long int	
	shortreal			signed	float	

Funcionamento do Circuito

- Descrição de como o circuito funciona;
- Abordagens:
  - Fluxo de Dados (RTL);
  - Hierárquica;
  - Comportamental;



Descrição por Fluxo de Dados

- Descreve o circuito em termos de expressões (lógicas e aritméticas) que relacionam entradas e saídas;
- Utiliza a diretiva **assign**;

```
assign <saida ou net> = (constante, segmento de vetor ou
                         expressão);
```

- Não pode-se utilizar o **assign** para mudar valores de dados do tipo **reg**.

Descrição por Fluxo de Dados

- **Atribuição de Constantes:**
  - Declaração:

```
<tamanho>'<base><valor>
```

  - Tamanho: Quantidade de *bits*;
  - Base:
    - `d` ou `D`: Decimal;
    - `h` ou `H`: Hexadecimal;
    - `b` ou `B`: Binário;
    - `o` ou `O`: Octal.

Descrição por Fluxo de Dados

- **Atribuição de Constantes:**
  - Exemplos:

```
assign x = 16’d255; //Decimal de 16 bits com valor 255
assign y = 8’hA0;   //Hexadecimal de 8 bits com valor A0
assign z = 4’b1001; //Binário de 4 bits com valor 1001
assign w = 6’o12;   //Octal de 6 bits com valor 12
```

Descrição por Fluxo de Dados

- **Atribuição de Constantes:**
  - Observações:
    - Quando não explicitado o tamanho e o tipo, suponha-se um valor do tipo **decimal** de **32 *bits***;

```
assign x = 12;      //Decimal de 32 bits com valor 12
```

    - Para representar valores negativos (em complemento de dois) coloca-se um sinal de menos **antes da especificação do tamanho**;

```
assign y = -8'd12;  //Decimal negativo de 8 bits (em
                    //complemento de dois)
```

Descrição por Fluxo de Dados

- **Atribuição de Constantes:**
  - Caracteres Especiais:
    - *Undeline* (_): pode ser utilizado para melhorar a legibilidade das atribuições;

```
assign x = 8'b1001_0011;  //Binário de 8 bits
```

    - *X ou x*: Valor desconhecido;

```
assign y = 8'hAx;          //Nibble menos significativo
                           //desconhecido
```

    - *Z ou z*: Alta impedância.

Descrição por Fluxo de Dados

- **Atribuição de um segmento de vetor:**
  - Declaração:

```
assign x = y[inicio:fim];
```

  - No caso, **x** tem tamanho `(fim-inicio)+1`;
  - Exemplos:

```
assign x = y[5:2];
assign z = y[1:0];
```

Descrição por Fluxo de Dados

- **Atribuição a partir de Expressões:**
  - Operadores Lógicos;
    - *Bit-wise*;
    - Redução;
    - Lógicos;
  - Operadores Aritméticos;
  - Operadores de Concatenação e Replicação;
  - Operadores de Deslocamento;
  - Operadores Relacionais;

Descrição por Fluxo de Dados

- **Operações Lógicas:**
  - Operadores *Bit-wise*:
    - São aplicados *bit*-a-*bit* dos operandos;

Operador	Operação Realizada	Exemplo: a = 4'b1010 b = 4'b0011
~	NOT	~a = 4'b0101
&	AND	a & b = 4'b0010
|	OR	a | b = 4'b1011
^	XOR	a ^ b = 4'b1001
~^ ou ^~	XNOR	a ~^ b = 4'b0110

Descrição por Fluxo de Dados

- **Operações Lógicas:**
  - Operadores *Bit-wise*:
    - Exemplos:

```
assign x = ~a;                  //O operador NOT admite
                                //apenas um operando!
assign y = a & b;
assign z = a | b;
assign q = a & b & c;
assign w = a ^ b;
assign r = a ~^ b;
```

Descrição por Fluxo de Dados

- **Operações Lógicas:**
  - Operadores *Bit-wise*:
    - Observações:
      - Os operadores *Bit-wise* valem mesmo que os operandos tenham apenas um *bit*;
      - A quantidade de *bits* da saída de um operador *Bit-wise* é igual a quantidade de *bits* do operando mais longo;
        - Preenchimento com zeros;

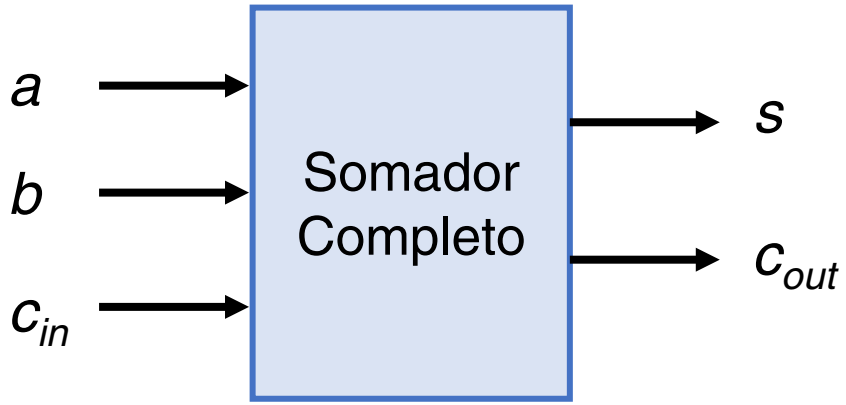
Descrição por Fluxo de Dados

Projeto 01 – Somador completo

Descreva o circuito corresponde ao somador completo em SystemVerilog utilizando a abordagem por fluxo de dados.

$$s = a \oplus b \oplus c_{in}$$

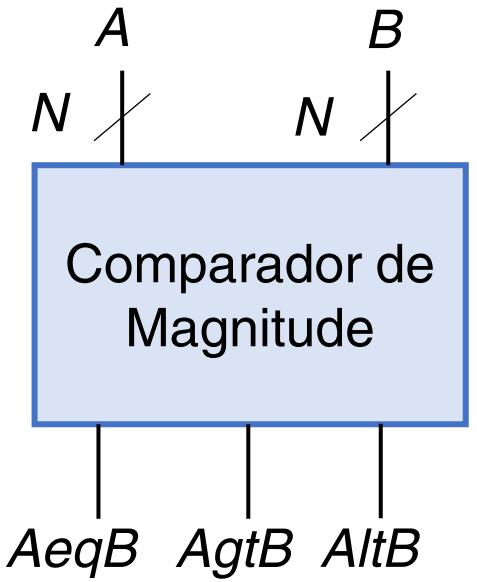
$$c_{out} = ab + ac_{in} + bc_{in}$$



Descrição por Fluxo de Dados

Projeto 02 – Comparadores de magnitude

Projete, utilizando o SystemVerilog, um circuito que se comporte como um comparador de magnitude de 4 *bits*. O circuito possui duas entradas, denominadas de *A* e *B*, ambas de 4 *bits*. O circuito possui três saídas, denominas de *AeqB*, *AgtB* e *AltB*, que indicam quando, respectivamente, *A*=*B*, *A*>*B* e *A*<*B*.



$$x_3 = (A_3 \oplus B_3)' = A_3'B_3' + A_3B_3$$
$$x_2 = (A_2 \oplus B_2)' = A_2'B_2' + A_2B_2$$
$$x_1 = (A_1 \oplus B_1)' = A_1'B_1' + A_1B_1$$
$$x_0 = (A_0 \oplus B_0)' = A_0'B_0' + A_0B_0$$

$$AeqB = x_3x_2x_1x_0$$

$$AgtB = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$$

$$AltB = A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1 + x_3x_2x_1A_0'B_0$$

Descrição por Fluxo de Dados

- **Operações Lógicas:**
  - Operadores de Redução:
    - Possuem apenas um operando e reduzem um vetor a um único *bit*;
    - A operação é feita com os *bits* do operando;

Operador	Operação Realizada	Exemplo: a = 4'b1010
&	AND	&a = 1'b0
~&	NAND	~&a = 1'b1
|	OR	|a = 1'b1
~|	NOR	~|a = 1'b0
^	XOR	^a = 1'b0
~^ ou ^~	XNOR	~^a = 1'b1

Descrição por Fluxo de Dados

- **Operações Lógicas:**
  - Operadores de Redução:
    - Exemplos:

```
assign x = &a;
assign y = ~&a;
assign z = |a;
assign q = ~|a;
assign w = ^a;
assign e = ~^a;
```

Descrição por Fluxo de Dados

- **Operações Lógicas:**
  - Operadores Lógicos:
    - Resultam em um único *bit*, independente da quantidade de *bits* das entradas;
    - Se o dado é diferente de zero, a saída é 1;
    - Se o dado é igual a zero, a saída é 0;

Operador	Operação Realizada	Exemplo: a = 3'b101 (=1b) b = 3'b000 (=0b)
!	NOT	!a = 1'b0 !b = 1'b1
&&	AND	a && b = 1'b0
||	OR	a || b = 1'b1

Descrição por Fluxo de Dados

- **Operações Lógicas:**
  - Operadores Lógicos:
    - Exemplos:

```
assign x = a && b;
assign y = a || b;
assign z = !a;
```

Descrição por Hierarquia

- Descreve o circuito pela associação de blocos previamente definidos, permitindo a construção de um sistema maior;
- Declaração:

```
<nome_do_modulo> nome(mapeamento);
```

  - **`nome_do_modulo`**: Nome do módulo a ser referenciado;
  - **`nome`**: nome da instância especifica do módulo;
  - **`mapeamento`**: forma que as entradas e saídas estão ligadas;
    - Lista;
    - Nome;

Descrição por Hierarquia

- **Associação por Lista:**
  - As entradas/saídas são colocadas na ordem no qual foram declaradas no modulo;
  - A ordem importa;
  - Exemplo:

```
module half_adder(a, b, s, c);
```

```
half_adder ha1 (a1, b1, s1, c1);
```

Descrição por Hierarquia

- **Associação por Nome:**
  - As entradas/saídas são associadas a pinos específicos do módulo;
  - A ordem não importa;
  - Exemplo:

```
module half_adder(a, b, s, c);
```

```
half_adder ha1 (.a(a1), .b(b1), .s(s1), .c(c1));
```

```
half_adder ha1 (.a(a1), .s(s1), .b(b1), .c(c1));
```

Descrição por Hierarquia

- **Observações:**
  - Assim como a abordagem estrutural, deve-se utilizar `wire` ou `logic` para fazer a ligação entre blocos;
  - Os módulos primários devem estar:
    - No mesmo projeto do módulo principal;
      - O módulo principal é aquele que é o *Top-Level*;
    - No mesmo arquivo do módulo principal;
      - O modulo principal é aquele com o nome do arquivo;

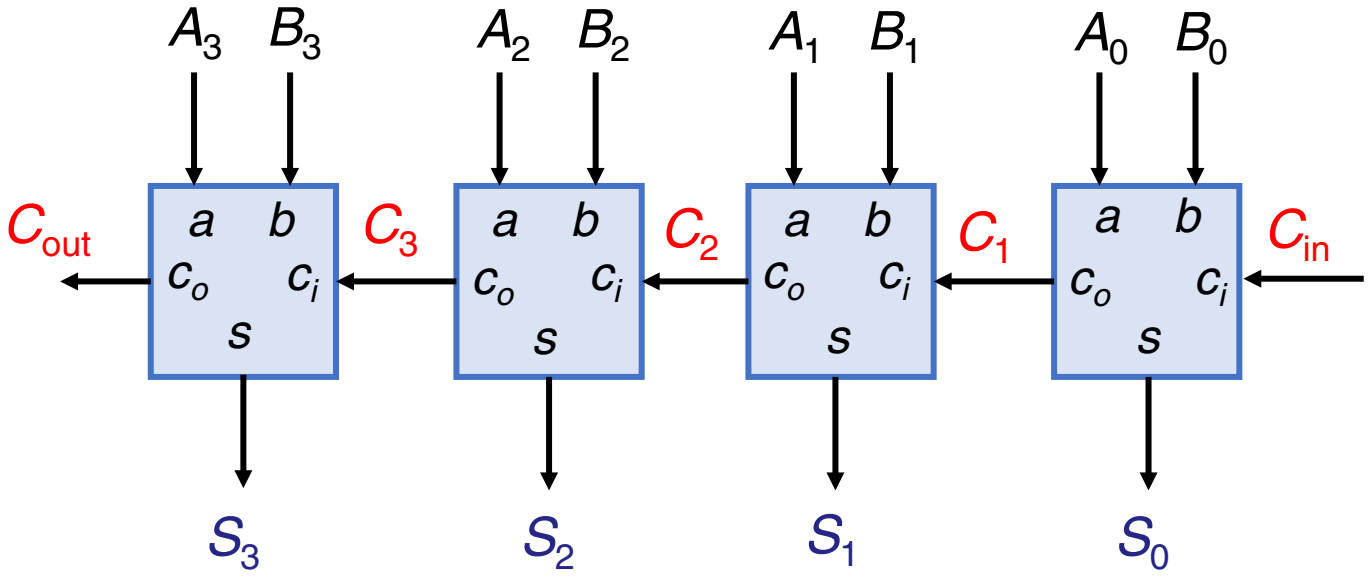
Descrição por Hierarquia

Projeto 03 – Somador de 4-bits

Projete um somador de 4-*bits* em SystemVerilog. Para isso, utilize a abordagem estrutural e hierarquia.

a) Inicie o seu projeto com um somador completo.

b) Utilizando o somador completo do item (a), implemente o somador de 4-*bits*.



Descrição por Fluxo de Dados (*continuação*)

- **Operações Aritméticas:**

Operador	Operação Realizada	Exemplo: a = 11 b = 5
+	Soma	a + b = 16
-	Subtração	a – b = 6
*	Multiplicação	a*b = 55
/	Divisão	a/b = 2
%	Resto da divisão	a%b = 1
**	Exponenciação	b**2 = 25

- O *bit* de *Carry-out* é gerado automaticamente;
  - Se somar dois números de 4 *bits*, o 5ª *bit* resultante da soma é o carry-out;

- Exemplos:

```
assign x = a + b;
assign y = a - b;
assign w = a * b;
assign z = a / b;
```

Descrição por Fluxo de Dados (*continuação*)

- **Operações de Concatenação e Replicação:**
  - Servem para juntar os operandos e formar um novo vetor;

Operador	Operação Realizada	Exemplo: a = 3'b011 b = 2'b01
{ }	Concatenação	{a,b} = 5'b01101
{{ }}	Replicação	{3{b}} = 6'b010101

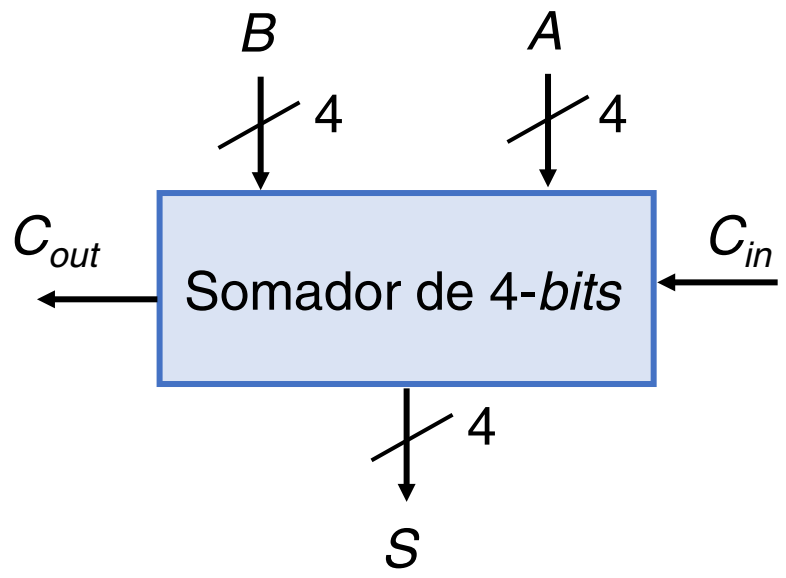
  - Exemplos:

```
assign x = {a, b};
assign y = {2{a}};
```

Descrição por Fluxo de Dados (*continuação*)

Projeto 04 – Somador de 4-bits (revisitado)

Projete um somador de 4-*bits* em SystemVerilog utilizando operações aritméticas.



Descrição por Fluxo de Dados (*continuação*)

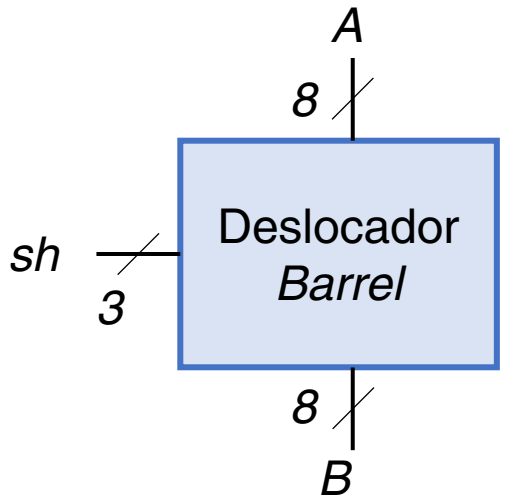
- **Operações de Deslocamento:**
  - Deslocamentos lógicos: todos os *bits* são deslocados;
  - Deslocamentos aritméticos: preserva-se o MSB e desloca-se os restantes dos *bits*;
  - *Bits* deslocados são perdidos.

Operador	Operação Realizada	Exemplo: a = 4'b0110 b = 4'b1011
<<	Deslocamento à esquerda lógico	a<<1 = 4'b1100
>>	Deslocamento à direita lógico	a>>1 = 4'b0011
<<<	Deslocamento à esquerda aritmético	a <<<1 = 4'b0100
>>>	Deslocamento à direita aritmético	b>>>1 = 4'b1101

Descrição por Fluxo de Dados (*continuação*)

Projeto 05 – Deslocadores Barrel

Projete um deslocador *barrel* em SystemVerilog. O deslocador em questão possui uma entrada *A*, de 8 *bits*, que indica o dado a ser deslocador e uma entrada *sh*, de 3 *bits*, que indica quantos deslocamentos devem ser dados ao número *A*. A saída é apresentada em *B*, também de 8 *bits*.



Descrição por Fluxo de Dados (*continuação*)

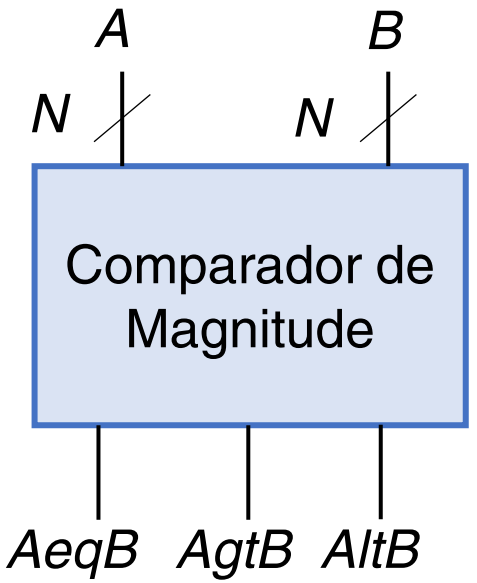
- **Operações Relacionais:**
  - São utilizados para comparar dois operandos;
  - Como resultado tem-se 0, quando a relação é falsa ou 1, quando a relação é verdadeira;

Operador	Operação Realizada	Exemplo: a = 4'b0110 b = 4'b0011 c = 4'b0011
>	Maior que	a > b = 1'b1
<	Menor que	a < b = 1'b0
==	Igual a	a==b = 1'b0
>=	Maior ou igual que	a >= b = 1'b1 a >= c = 1'b1
<=	Menor ou igual que	a <= b = 1'b0
!=	Diferente	a != b = 1'b1

Descrição por Fluxo de Dados

Projeto 06 – Comparadores de magnitude (revisitado)

Projete, utilizando o SystemVerilog, um circuito que se comporte como um comparador de magnitude de 4 *bits*. O circuito possui duas entradas, denominadas de *A* e *B*, ambas de 4 *bits*. O circuito possui três saídas, denominas de *AeqB*, *AgtB* e *AltB*, que indicam quando, respectivamente, *A*=*B*, *A*>*B* e *A*<*B*. No seu projeto, utilize os operadores relacionais.



Descrição por Fluxo de Dados (*continuação*)

- **Atribuição Condicional:**
  - Corresponde a um ternário em C;

```
assign x = (condição) ? (verdadeiro) : (falso);
```

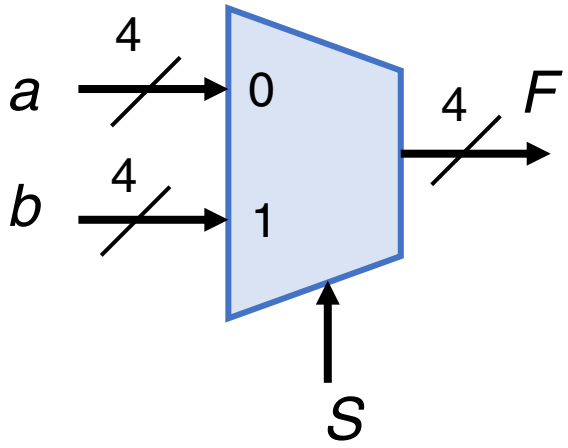
  - Exemplo:

```
assign x = (sel==1'b1) ? 4'b0010 : 4'b1100;
   //Verifica sel.
   //Se sel = 1 (verdadeiro), então x = 4'b0010.
   //Se sel = 0 (falso), então x = 4'b1100.
```

Descrição por Fluxo de Dados (*continuação*)

Projeto 07 – Multiplexador 2x1 de 4-bits

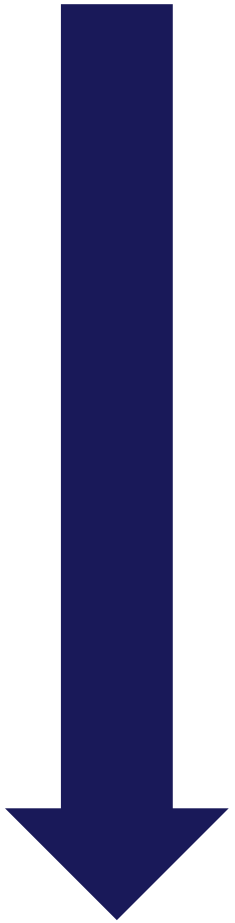
Descreva o circuito correspondente a um multiplexador 2x1 em SystemVerilog utilizando a abordagem por fluxo de dados e utilizando atribuição condicional.



Descrição por Fluxo de Dados (*continuação*)

- Prioridade dos Operadores:

Operador	Prioridade
Operadores unários	
**	
* / %	
+ -	
<< >> <<< >>>	
< > <= >=	
== ~= === ~==	
& (operador binário)	
^ ~^ ^~ (operador binário)	
| (operador binário)	
&&	
||	
?:	
{} {{}}	



Descrição Comportamental

- Descreve o circuito em termos do seu comportamento (algoritmo);
- A descrição comportamental deve ser especificada em **procedimentos**;
- O procedimento tem (geralmente) uma lista de sensibilidade;
- Somente quando alguma variável da lista de sensibilidade muda é que o procedimento executa.
- Um módulo pode conter quantos procedimentos for necessário;
- Os procedimentos não podem ser alinhados (um procedimento não pode conter outro);
- Todos os procedimentos são executados de forma paralela;

Descrição Comportamental

- **Tipos de Procedimentos:**
  - `initial`:
    - Executado apenas uma vez;
    - Utilizado para inicialização de variáveis ou monitoramento (bancada de testes);
  - `always`:
    - Executado constantemente;
    - Utilizado para descrever o funcionamento de um circuito;

Descrição Comportamental

- **Procedimento `initial`:**
  - Declaração:

```
initial begin
    (...)
end
```

  - Não possui lista de sensibilidade;

Descrição Comportamental

- **Procedimento `always`:**
  - Declaração:

```
always @(lista_de_sensibilidade) begin
    (...)
end
```

  - Os procedimentos `always` podem ser de dois tipos:
    - Combinacionais;
    - Sensíveis ao *Clock*;
  - Observação: O `begin` e o `end` são necessários apenas se o procedimento tiver mais de uma linha;

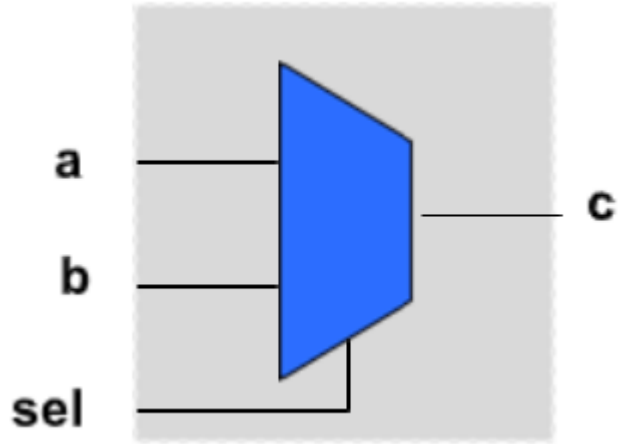
Descrição Comportamental

- **Procedimento `always`:**
  - Observação: Nos procedimentos sensíveis ao *Clock*, utiliza-se duas diretivas:
    - `posedge:` Sensível a borda de subida;
    - `negedge:` Sensível a borda de descida;

Descrição Comportamental

- **Procedimento `always`:**
  - Procedimento Combinacional: Sensível a todas as entradas;

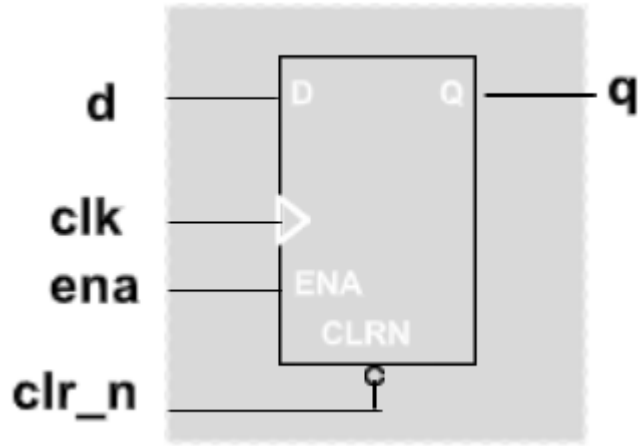
```
always @(a, b, sel)
always @*
```



  - Procedimento sensível ao *Clock*: Sensível ao *Clock* ou ao sinais de controle;

```
always @(posedge clk, negedge clr_n)
```

Não inclui as entradas d e ena!



Descrição Comportamental

- **Procedimento `always`:**
  - Dentro do procedimento pode-se realizar atribuições de variáveis;
    - A variável que recebe o valor da expressão deve ser do tipo `reg` ou `logic`;
    - Não há o termo `assign`;
    - Só é feita quando o procedimento é executado.
  - Observação:
    - Quando utiliza-se o termo `assign`, a atribuição é contínua;
      - Quando um dos termos da expressão muda, a saída muda ao mesmo tempo;
    - Quando utiliza-se uma atribuição dentro de um procedimento, ela é procedural;
      - A saída só muda quando o procedimento é executado.

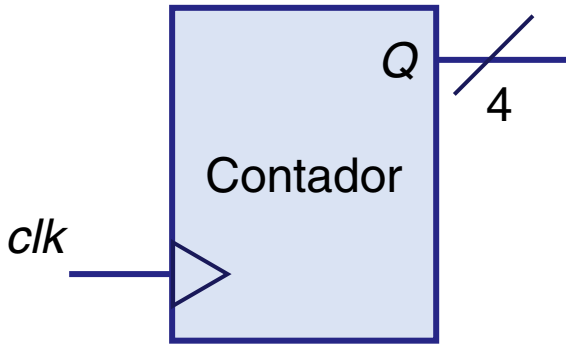
Descrição Comportamental

- **Procedimento `always_comb`:** Serve para implementar apenas circuitos combinacionais;
- **Procedimento `always_ff`:** Serve para implementar circuitos sequências gatilhados pela borda;
- **Procedimento `always_latch`:** Serve para implementar circuitos sequências gatilhados pelo nível (*latches*);

Descrição Comportamental

Projeto 08 – Contador de 4-bits

Descreva um contador de 4 *bits* em Verilog utilizando a abordagem comportamental. O contador é sensível a borda de subida do relógio.



Descrição Comportamental

- Dentro de um procedimento (e somente dentro de um), pode-se utilizar estruturas de decisão, seleção ou de repetição;
  - Estrutura de seleção: ***case***, ***casez***, ***casex***;
  - Estrutura de decisão: ***if-else if-else***;
  - Estruturas de repetição: ***repeat*** e ***for***;

Descrição Comportamental

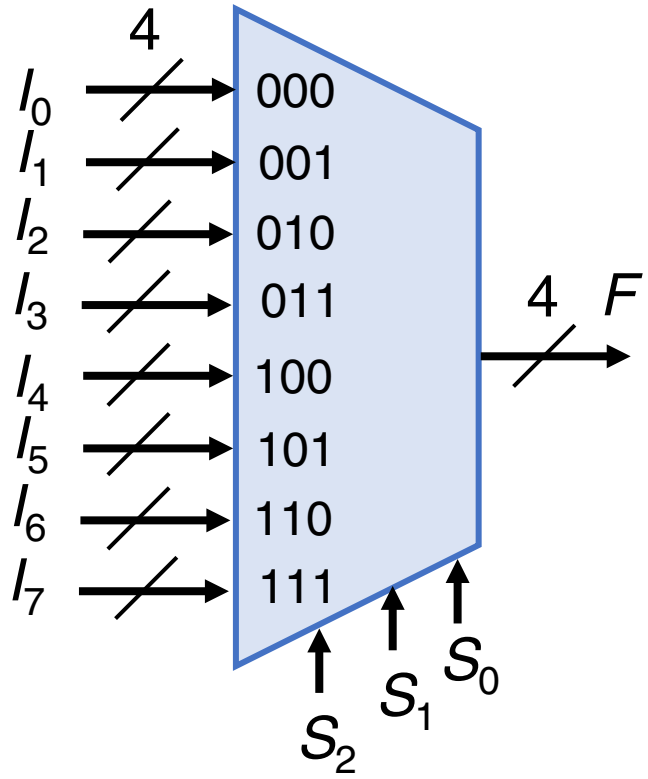
- **Estrutura *case*:**
  - Declaração:

```
case (identificador)
    (valor1): begin
        (...)
    end
    (valor2): begin
        (...)
    end
    default: begin      //opcional
        (...)
    end
endcase
```

Descrição Comportamental

Projeto 09 - Multiplexador 8x1 de 4-bits

Descreva um Multiplexador 8x1 de 4 *bits* em SytemVerilog utilizando a abordagem comportamental.



Descrição Comportamental

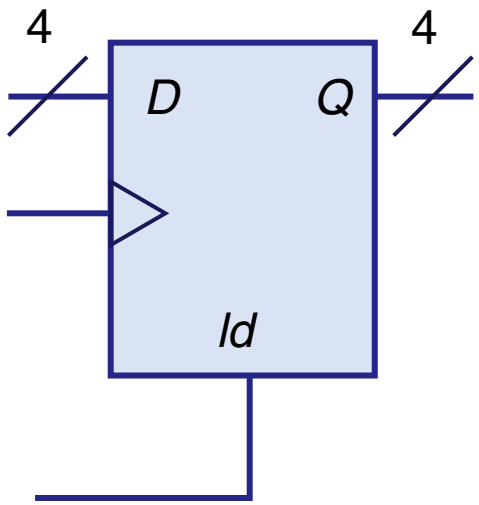
- **Estrutura *if-else if-else*:**
  - Declaração:

```
if (condição_1) begin
    (...)
end
else if(condição_2) begin
    (...)
end
    (...)
else begin
    (...)
end
```

Descrição Comportamental

Projeto 10 – Registrador de carga paralela

Descreva um registrador de carga paralela de 4 *bits* em SystemVerilog utilizando a abordagem comportamental. O registrador possui uma entrada *ld*, que quando *ld* = 1, deve-se efetuar a carga no registrador. Caso contrário (*ld* = 0), o último valor é mantido. O registrador é sensível à borda de descida do relógio.



Descrição Comportamental

- **Estrutura *casex*:**
  - Adiciona-se a possibilidade de utilizar os valores ‘?’, ‘x’ ou como não interessa (*don’t-care*);
  - Declaração:

```
casex (identificador)
    (valor1): begin
        (...)
    end
    (valor2): begin
        (...)
    end
    default: begin      //opcional
        (...)
    end
endcase
```

Descrição Comportamental

- **Estrutura *for*:**
  - Especifica:
    - Uma condição inicial;
    - Uma condição de parada;
    - Uma condição de iteratividade.
  - Declaração:

```
for (cond_inicial; cond_parada; incremento) begin
    (...)
end
```

  - Observação: Geralmente utiliza-se uma variável do tipo `integer` para o contador do laço;

Descrição Comportamental

- Estrutura ***repeat***:
  - Executa uma quantidade fixa de repetições;
  - Declaração:

```
repeat (vezes) begin
    (...)
end
```

  - Exemplo:

```
if(rotate == 1) begin
    repeat (8) begin
        temp = data[15];
        data = {data << 1, tmp};
    end
end
```

Descrição Comportamental

- **Atribuição Bloqueável vs. Atribuição Não-Bloqueável:**
  - Dentro de um procedimento, existem dois tipos de atribuição:
    - Bloqueável (=);
    - Não-Bloqueável (<=);
  - Atribuição Bloqueável (=):
    - São executadas na ordem em que são especificadas;
    - Se existem duas designações de bloqueamento consecutivas, a segunda designação não pode ser executada até que a primeira tenha finalizado;
  - Atribuição Não-Bloqueável (<=):
    - São executadas ao mesmo tempo;
    - Se existem duas designações de não-bloqueamento consecutivas, a segunda designação é executada ao mesmo tempo em que a primeira.

Descrição Comportamental

- **Atribuição Bloqueável vs. Atribuição Não-Bloqueável:**
  - Pode-se utilizar atribuições bloqueáveis e não-bloqueáveis em um mesmo procedimento, porém, **em alguns casos**, elas levam a comportamentos diferentes;
  - Regra de Ouro:
    - Utilize a atribuição Bloqueável (=) para lógica combinacional;
    - Utilize a atribuição não-Bloqueável (<=) para lógica sequencial;
    - Sempre verifique o *Hardware* gerado de forma a verificar o comportamento;

Descrição Comportamental

- **Atribuição Bloqueável vs. Atribuição Não-Bloqueável:**
  - Exemplo:

```
module blocking (clk, x, y);
     input logic clk;
     input logic [7:0] x;
     output logic [7:0] y;

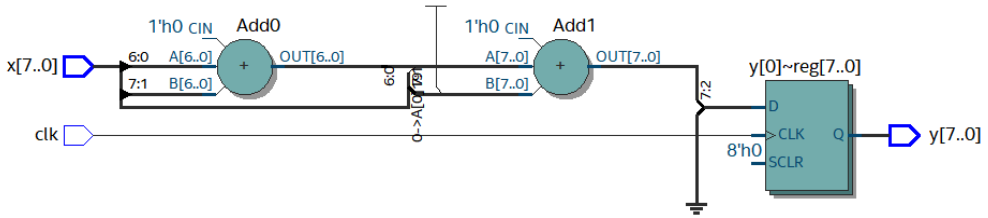
     logic [7:0] x1, x2, x3, x4;

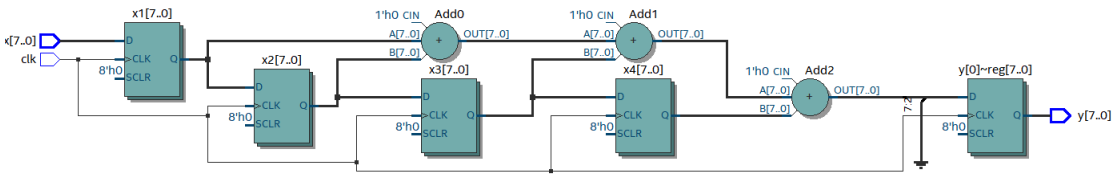
     always @(posedge clk) begin
          x1 = x;
          x2 = x1;
          x3 = x2;
          x4 = x3;
          y = (x1 + x2 + x3 + x4) >> 2;
     end
endmodule
```

```
module non-blocking (clk, x, y);
     input logic clk;
     input logic [7:0] x;
     output logic [7:0] y;

     logic [7:0] x1, x2, x3, x4;

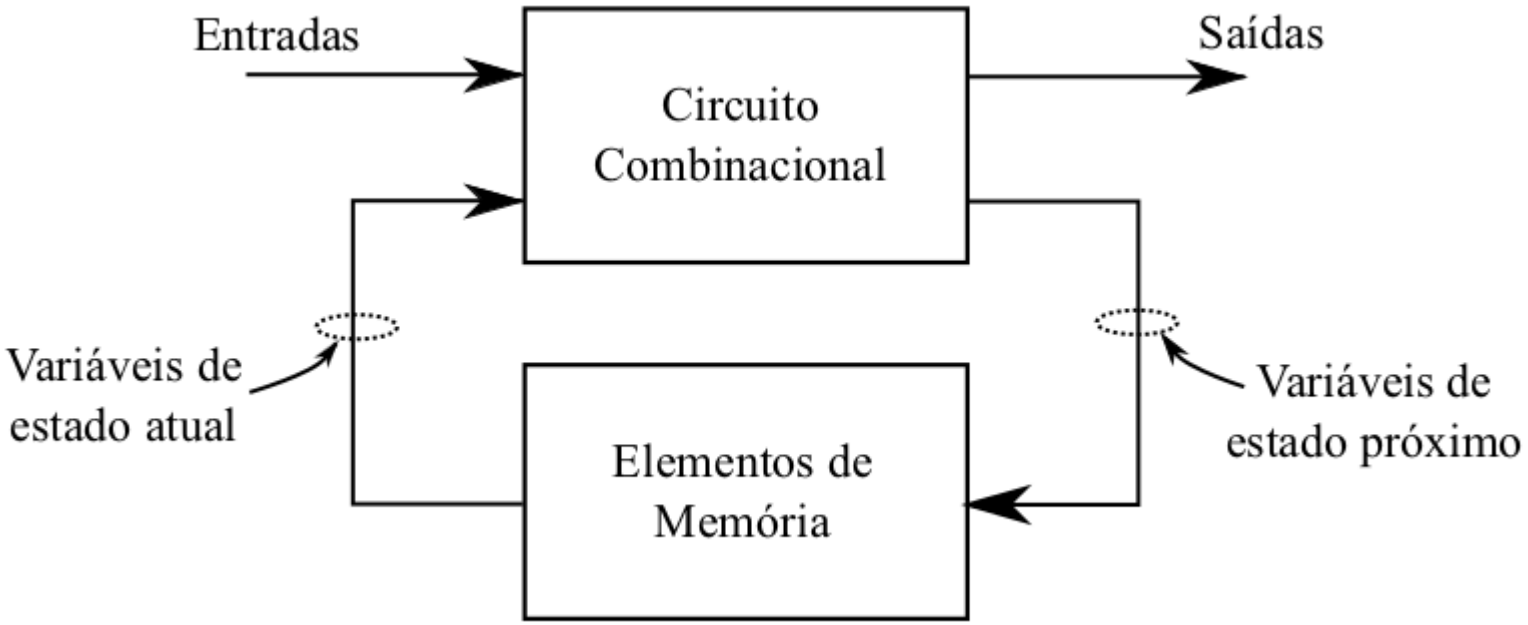
     always @(posedge clk) begin
          x1 <= x;
          x2 <= x1;
          x3 <= x2;
          x4 <= x3;
          y <= (x1 + x2 + x3 + x4) >> 2;
     end
endmodule
```





Máquina de Estados em SystemVerilog

- Estrutura Geral para Máquinas de Estados:
  - Circuito Combinacional;
  - Elementos de Memória (Registrador de Estados);



Máquina de Estados em SystemVerilog

- Os estados devem ser codificados e declarados como parâmetros ou como enumeráveis;

```
typedef enum {idle, state0, state1, state2} estado;
        estado currState, nextState;
```

- São necessários dois procedimentos;
  - Primeiro procedimento:
    - Sensibilidade: Sinais de *Clock* e de *Reset*;
    - Analisa o *estado presente* e as *entradas* para gerar o *estado seguinte*;
    - Deve-se utilizar um `case`, com identificador de variável de estado, para definir o estado seguinte;
    - Pode-se utilizar um sinal de *Reset* assíncrono.

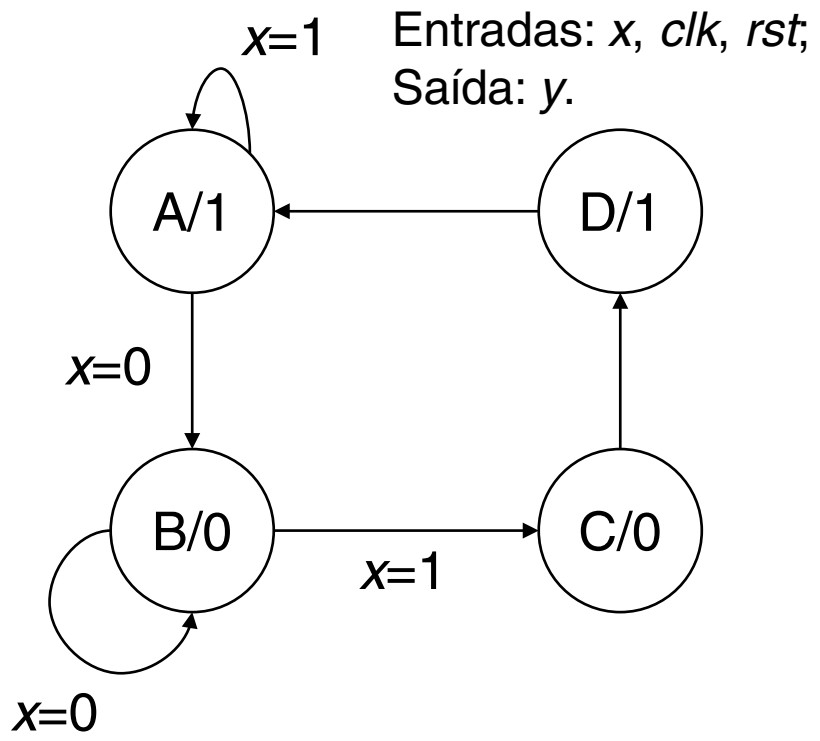
Máquina de Estados em SystemVerilog

- Segundo procedimento:
  - Analisa o estado presente e as entradas (caso Mealy) para gerar a *saída*;
  - Sensibilidade:
    - Máquina Moore → Estado Atual;
    - Máquina Mealy → Estado Atual e Entrada;
  - Deve-se utilizar um `case`, com identificador de variável de estado, para definir a saída;

Máquina de Estados em SystemVerilog

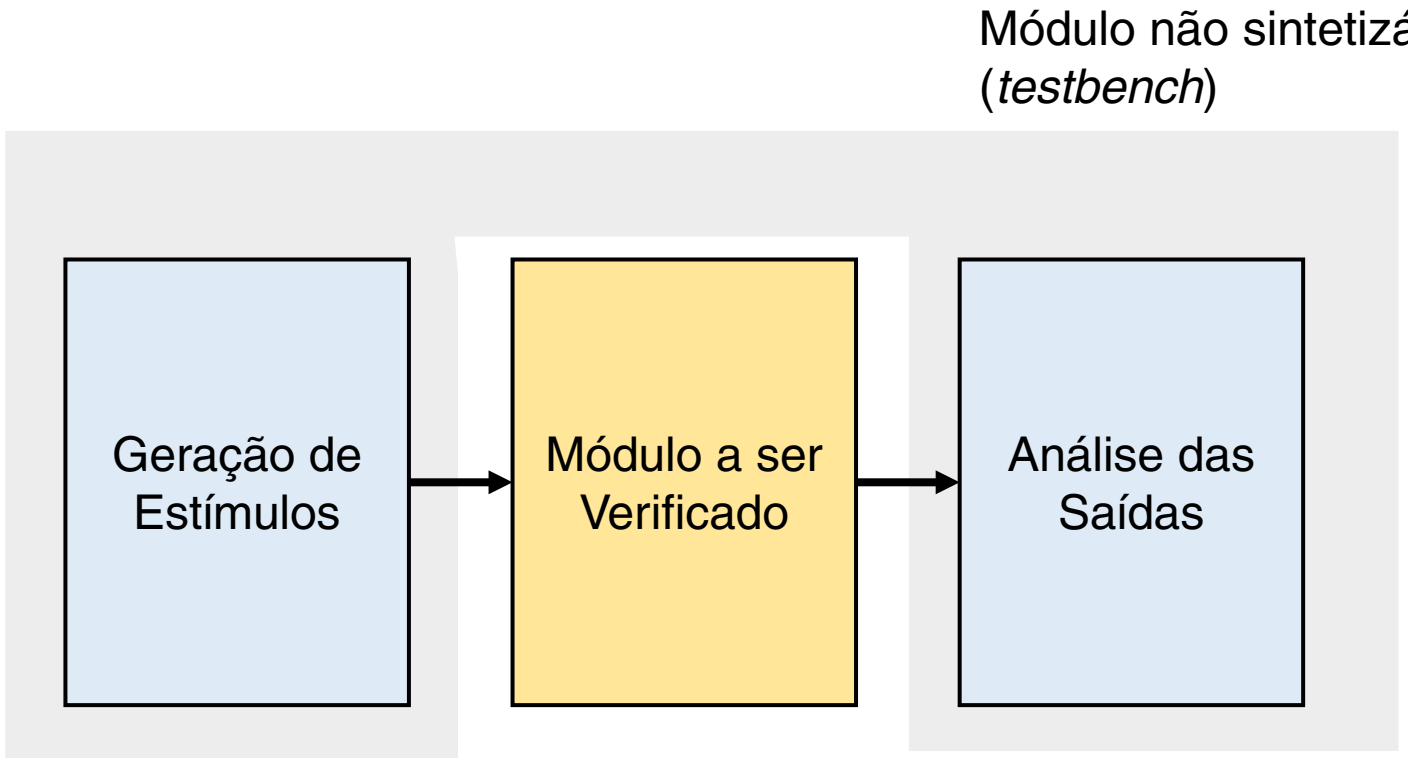
Projeto 11 – Máquina de estados em Verilog

Implemente, em Verilog, a máquina de estados com o diagrama de transição apresentado na Figura ao lado.



Introdução à Verificação

- *Testbech*: *Script* automatizado para simulação (verificação) de módulos descritos em linguagem de descrição de *hardware*;
- Visão geral de um *testbench*:



Introdução à Verificação

- Os *testbench* não possuem entradas;
- O módulo a ser testado deve ser instanciado no *testbench*;
- Deve-se declarar as entradas e saídas como dados intermediários do tipo **`logic`**;
- Utilizar o procedimento **`initial`** para atribuir valores à entradas;
  - Utiliza o *delay* temporal (`#delay`) para impor a temporização das entradas;
  - Usar a diretiva **`$stop`** ou **`$finish`** para encerrar a simulação;
- Utilizar o procedimento **`always`** para variáveis que devem ser modificadas constantemente (ex. *clock*);
- Utilizar a diretiva **`$display`** ou **`$monitor`** para apresentar resultados na tela;
- A base de tempo e a resolução é ajustada pela diretiva **`` `timescale ``**.